

iOS组件化方案-总结第一篇



sun6boys (/u/3e8d48aaab02) [+关注](#)

2016.12.19 00:44* 字数 1675 阅读 14836 评论 64 喜欢 136 赞赏 1

(/u/3e8d48aaab02)

概述

近一年iOS业界讨论组件化方案甚多，大体来说有3种。

- Protocol注册方案
- URL注册方案
- Target-Action runtime调用方案

URL注册方案据我了解很多大公司都在采用，蘑菇街 App 的组件化之路 (<https://link.jianshu.com?t=http://limboy.me/tech/2016/03/10/mgj-components.html>)蘑菇街的Limboy在这篇博客中做了很详尽的阐述

Target-Action runtime调用方案Casa在 iOS应用架构谈 组件化方案 (<https://link.jianshu.com?t=http://casatwy.com/iOS-Modulization.html>)中也做了很详尽的描述，前阵时间Casa开了一篇博客在现有工程中实施基于CTMediator的组件化方案 (https://link.jianshu.com?t=http://casatwy.com/modulization_in_action.html)清楚讲述了如何用这套方案实施组件化

Protocol方案我尚未看到有人做过详尽的分享，也许是我孤陋寡闻，不过在这里，我会教大家用Protocol方案实施组件化,不仅如此..

我会采用以上3种方案详尽的实现3个Demo，Demo会在文尾给到，本文不过多阐述3种方案的优劣，我会在最后做一个总结，希望给想了解组件化方案的同学或者给在项目中准备实施组件化方案的同学提供一个借鉴。

第二篇 也已经出来了传送门iOS组件化方案-总结第二篇
(<https://www.jianshu.com/p/a5dfd986bfa7>)

业务模拟场景

- 首页 展示商品列表
- 商品详情页 展示商品的详细信息
- 确认订单页 生成订单

把业务连贯起来 点击 首页 中A商品，进入A商品的 商品详情页，点击 商品详情页 中的 立即购买 进入 确认订单页，点击 确认订单页 中的提交订单 会返回到 商品详情页，并且在 商品详情页 中告知用户下单成功。

真实业务场景下 确认订单页 点提交订单 是不会回到 商品详情页 的，模拟这个场景是想在Demo中实现2个模块中反向回调。

(/apps/download?utm_source=sbc) ×

一、Protocol注册方案

正式实施前先奉上**Demo**,建议只下一个主项目就可以了（注：下载完不需要pod install或者pod update， pods在我私有源上 我没有填写.gitignore文件,下载完都是可以直接跑的）

主项目地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRMainProject>)

商品详情业务接口组件地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRGoodsDetailServiceProtocol>)

商品详情业务组件地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRGoodsDetail>)

确认订单业务接口组件地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRConfirmOrderServiceProtocol>)

确认订单业务组件地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRConfirmOrder>)

业务调度中间件地址 (<https://link.jianshu.com?t=https://github.com/sun6boys/CRProtocolManager>)

1.基本准备工作

- 先去gitHub创建一个项目存放私有Repo源,repo地址 <https://github.com/sun6boys/CRRepositories.git> 后面3种方案私有pod源都会放在这里。
- 本地添加私有源 终端执行命令 `pod repo add CRRepositories https://github.com/sun6boys/CRRepositories.git` （如果之前并未向gitHub push过文件也没有把SSH公钥保存到gitHub，这时候应该会提示你输入gitHub账号密码）
- 以上操作完成 `cd ~/.cocoapods/repos` 目录下至少会有2个文件夹 **CRRepositories** 和 **master**, **master**文件下面存放的是公有源文件,CRRepositories*目录下目前是空的，后面会存放我们私有源文件
- 基本准备工作完成。

2.Xcode创建项目[CRProtocolManager]

CRProtocolManager和**MGJRouter**、**CTMediator**一样属于模块之间调度的中间件

在**CRProtocolManager**项目下创建名为**CRProtocolManager**的文件夹，后面我们需要做成私有pod的文件均放在该文件夹下。

创建**CRProtocolManager**类(.h,.m)，定义2个对外接口

```
@interface CRProtocolManager : NSObject

+ (void)registerServiceProvide:(id)provide forProtocol:(Protocol*)protocol;

+ (id)serviceProvideForProtocol:(Protocol *)protocol;

@end
```

具体方法实现很简单可以参看Demo，我这里只是简单处理。

接下来就是要把项目提交到gitHub，做私有pod了

- gitHub新建一个project名为**CRProtocolManager**
- 终端cd至**CRProtocolManager**项目目录下执行命令 `git remote add origin https://github.com/sun6boys/CRProtocolManager.git`
- 因cocoaPods强制添加开源许可文件执行命令 `echo MIT>FILE_LICENSE` 创建名为**FILE_LICENSE**的文件
- 终端cd至**CRProtocolManager**目录下执行命令 `pod spec create CRProtocolManager`
- 执行命令 `vim .CRProtocolManager.podspec` 编辑podspec文件，具体如何编辑可参看Demo中的podspec文件或者google
- 退出编辑执行命令 `git add .`
- ``git commit -m 'log'`
- `git tag 0.0.1` **tag**一定要和podspec中的version一致
- `git push origin master --tags --tags`为了把刚才添加的tag提交上去
- 执行命令 `pod repo push CRRepositories CRProtocolManager.podspec --verbose --allow-warnings` 注：**CRRepositories**即为准备工作中的私有源仓库
- 成功后 `pod search CRProtocolManager` 应该就能搜索到了

万里长征终于走完第一步，基础设施已经构建完毕

3.商品详情业务模块

既然组件化了，那我们所有的业务模块都是单独的project，但是这里我会分2个project，一个是商品详情业务入口模块，一个是商品详情业务模块。业务入口模块即是定义该模块对外提供业务接口的protocol，如果A模块需要调用到B模块，那A模块只需要引入CRProtocolManager和B模块的protocol，而不是引入整个B模块。

新建一个project **CRGoodsDetailServiceProtocol**，创建一个和项目名一样的protocol文件，定义接口如下

```
@protocol CRGoodsDetailServiceProtocol <NSObject>

@required;

- (UIViewController *)goodsDetailViewControllerWithGoodsId:(NSString*)goodsId goodsName:(NSString *)goodsName;

@end
```

参照**CRProtocolManager**做成私有pod

以上实施完毕，新建一个project **CRGoodsDetail**，新建2个类

```
CRGoodsDetailServiceProvide
CRGoodsDetailViewController
```

CRGoodsDetailServiceProvide 即是 CRGoodsDetailServiceProtocol 的实现者 所以他依赖 CRGoodsDetailServiceProtocol , 因为商品详情模块需要跳转到订单确认页, 所以他也依赖 CRProtocolManager 。

添加Podfile文件编辑如下

```
source 'https://github.com/sun6boys/CRRpositories.git'
source 'https://github.com/CocoaPods/Specs.git'

target 'CRGoodsDetail' do

  pod "CRProtocolManager"
  pod "CRGoodsDetailServiceProtocol"

end
```

执行 `pod install --verbose --no-repo-update`

最终 CRGoodsDetailServiceProvide 实现代码如下

```
#import "CRGoodsDetailServiceProvide.h"
#import <CRGoodsDetailServiceProtocol/CRGoodsDetailServiceProtocol.h>
#import <CRProtocolManager/CRProtocolManager.h>

#import "CRGoodsDetailViewController.h"

@interface CRGoodsDetailServiceProvide()<CRGoodsDetailServiceProtocol>

@end

@implementation CRGoodsDetailServiceProvide

+ (void)load
{
    [CRProtocolManager registServiceProvide:[self alloc] initWithProtocol:@protocol(CRGoodsDetailServiceProtocol)];
}

- (UIViewController *)goodsDetailViewControllerWithGoodsId:(NSString *)goodsId goodsName:(NSString *)goodsName
{
    CRGoodsDetailViewController *goodsDetailVC = [[CRGoodsDetailViewController alloc] initWithGoodsId:goodsId goodsName:goodsName];
    return goodsDetailVC;
}

@end
```

CRGoodsDetailViewController 实现代码如下

```
#import "CRGoodsDetailViewController.h"

@interface CRGoodsDetailViewController ()

@property (nonatomic, copy) NSString *goodsId;
@property (nonatomic, copy) NSString *goodsName;

@property (nonatomic, strong) UILabel *statusLabel;
@property (nonatomic, strong) UIButton *buyButton;
@end

@implementation CRGoodsDetailViewController

- (instancetype)initWithGoodsId:(NSString *)goodsId goodsName:(NSString *)goodsName
{
    self = [super init];
    if (self) {
        _goodsId = goodsId;
        _goodsName = goodsName;
    }
    return self;
}

- (void)viewDidLoad {
    [super viewDidLoad];
    self.navigationItem.title = self.title;

    [self.view addSubview:self.statusLabel];
    [self.view addSubview:self.buyButton];
}

- (void)viewWillLayoutSubviews
{
    [super viewWillLayoutSubviews];
    self.statusLabel.frame = CGRectMake(0, 0, 100, 20);
    self.statusLabel.center = self.view.center;

    self.buyButton.frame = CGRectMake(0, self.view.frame.size.height - 45, self.view.frame.size.width, 45);
}

#pragma mark - event
- (void)didClickBuyButton:(UIButton *)button
{
}

#pragma mark - getters
- (UILabel *)statusLabel
{
    if (_statusLabel == nil) {
        _statusLabel = [[UILabel alloc] init];
        _statusLabel.textColor = [UIColor redColor];
        _statusLabel.font = [UIFont systemFontOfSize:15.f];
        _statusLabel.textAlignment = NSTextAlignmentCenter;
        _statusLabel.text = @"暂未购买";
    }
    return _statusLabel;
}

- (UIButton *)buyButton
{
    if (_buyButton == nil) {
        _buyButton = [UIButton buttonWithType:UIButtonTypeCustom];
        [_buyButton setTitle:@"立即购买" forState:UIControlStateNormal];
        [_buyButton setTitleColor:[UIColor whiteColor] forState:UIControlStateNormal];
        [_buyButton setBackgroundColor:[UIColor redColor]];
        [_buyButton addTarget:self action:@selector(didClickBuyButton:) forControlEvents:UIControlEventTouchUpInside];
    }
    return _buyButton;
}

@end
```

把 CRGoodsDetail 做成私有pod 记得编辑podspec文件的时候添加
dependency CRProtocolManager CRGoodsDetailServiceProtocol

4.新建主项目 MainProject

为了少建一个项目 首页 模块我是直接放在主项目中的, 按理 首页 也应该是一个独立的pod.

首页 业务场景是, 显示商品列表, 点击某个商品进入该商品详情页. 所以他依赖 CRGoodsDetailServiceProtocol 和 CRProtocolManager 因为首页模块即是主项目所以他还得依赖 CRGoodsDetail

最终首页核心代码如下

```
#pragma mark - event
- (void)didClickGoodsButton:(UIButton *)button
{
    id<CRGoodsDetailServiceProtocol> goodsServiceProvide = [CRProtocolManager serviceProvideForProtocol:@protocol(CRGoodsDetailServiceProtocol)];
    UIViewController *goodsDetailVC = [goodsServiceProvide goodsDetailViewControllerWithGoodsId:@"123" goodsName:@"农夫山泉矿泉水"];
    [self.navigationController pushViewController:goodsDetailVC animated:YES];
}
```

5.确认订单模块

参照 商品详情 新建确认订单业务入口pod 以及确认订单业务pod.和 商品详情 有区别的是, 提交订单完成后要回到 商品详情 并且通知 商品详情 用户已经购买,所以 CRConfirmOrderServiceProtocol 接口定义如下

```
@protocol CRConfirmOrderServiceProtocol <NSObject>

- (UIViewController *)confirmOrderViewControllerWithGoodsId:(NSString *)goodsId sureComplete:(dispatch_block_t)sureComplete;

@end
```

最后记得在 商品详情 加上跳转并且podspec里面加上dependency

Protocol注册方案完结

初稿未完结 待续

小礼物走一走，来简书关注我

赞赏支持



(/u/ac93c4b98c6b)

📅 日记本 (/nb/7954762)

📄 举报文章 © 著作权归作者所有



sun6boys (/u/3e8d48aaab02) ♂
写了 5134 字, 被 474 人关注, 获得了 222 个喜欢
(/u/3e8d48aaab02)

+ 关注

喜欢 | 136



更多分享

(<http://cwb.assets.jianshu.io/notes/images/7764920>)



(/apps/download?utm_source=nbc)

被以下专题收入，发现更多相似内容



RN/链式编程... (/c/bd231d1508b3?utm_source=desktop&utm_medium=notes-included-collection)



iOS程序猿 (/c/527cd13a0f17?utm_source=desktop&utm_medium=notes-included-collection)



iOS备忘录 (/c/536499c136f6?utm_source=desktop&utm_medium=notes-included-collection)



iOS Dev... (/c/3233d1a249ca?utm_source=desktop&utm_medium=notes-included-collection)



@IT·互联网 (/c/V2CqjW?utm_source=desktop&utm_medium=notes-included-collection)



移动前沿 (/c/5aac963ca52d?utm_source=desktop&utm_medium=notes-included-collection)



iOS Col... (/c/1bc08101434b?utm_source=desktop&utm_medium=notes-included-collection)

展开更多

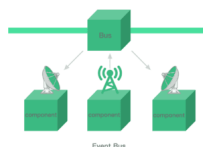
iOS 组件化方案，带有源码 (/p/225db657c4f0?utm_campaign=maleski...)

概述 近一年iOS业界讨论组件化方案甚多，大体来说有3种。Protocol注册方案 URL注册方案 Target-Action runtime调用方案 URL注册方案据我了解很多大公司都在采用，蘑菇街 App 的组件化之路蘑菇街的Limboy在



yuditxj (/u/712ceb5e4b8a?utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

([/p/9239669c3a96?](/p/9239669c3a96?utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)



/p/9239669c3a96?utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)

iOS开发——组件化及去Mode化方案 (/p/9239669c3a96?utm_campaign=maleski...)

1.组件化的目的是什么？最近一两年很多人都想在项目里面搞组件化，觉得搞组件化好，却鲜有人知道组件化好在哪里？组件化的目的是什么？个人觉得组件化主要有两个目的：1.实现项目代码的高内聚低耦合；2.方



CatherinePlans (/u/f796a54054d1?utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/fc13a4785ab8?



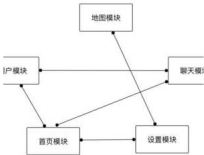
utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
iOS架构学习篇——组件化架构漫谈 (/p/fc13a4785ab8?utm_campaign=...

该文章属于<简书 — 刘小壮>原创，转载请注明：<简书 — 刘小壮>
<http://www.jianshu.com/p/67a6004f6930> 前段时间公司项目打算重构，准确来说应该是按之前的产品逻辑

 YIart (/u/56ec731dbda4?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/41a7e73fdabe?



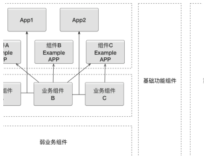
utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
经验分享 | 超详细的组件化架构方案 (/p/41a7e73fdabe?utm_campaign=...

文 / 刘小壮 (简书作者投稿) 原文链接: <http://www.jianshu.com/p/67a6004f6930> 前段时间公司项目打算重构，准确来说应该是按之前的产品逻辑重写一个项目😂。在重构项目之前涉及到架构选型的问题，我和组

 蓝鸥科技 (/u/bb62f66101d7?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/48fbcbb36c75?



utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
iOS App组件化开发实践 (/p/48fbcbb36c75?utm_campaign=maleskine...

前因 其实我们这个7人iOS开发团队并不适合组件化开发。原因是因为性价比低，需要花很多时间和经历去做这件事，带来的收益并不能彻底改变什么。但是因为有2~3个星期的空档期，并不是很忙；另外是可以用在

 曹俊_413f (/u/d8a653fc1cb1?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

月影 (/p/4f5bed9b3343?utm_campaign=maleskine&utm_content=not...

孤身只影

 宾那 (/u/dc644590c4d6?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/9541fb3bf981?



utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
Day58-- 【朋友圈是记录，也是纪念】 (/p/9541fb3bf981?utm_campaign=...

自从踏上微商这条路，添加的人越来越多，发的广告也越来越多。可能，很多小伙伴们都把我屏蔽掉了吧！好像一提到朋友圈，尽是负面措辞：影响生活质量、让人极度不自律、秀晒炫满天飞、微商代购.....关闭朋

 卖小妞的饼干 (/u/722785af67d9?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/9d40f43a2819?



utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
炫车宝OL 2.0版本全新登场 (/p/9d40f43a2819?utm_campaign=maleski...

微炫客炫车宝OL 2.0版本上线啦！现版本新增三大功能，并对六个功能入口进行了优化升级。此次更新，不仅能为商户拓展客源，也为商户提供更细致的管理模板，为后续商家更方便的管理自己的门店及平台打下基

 小炫宝 (/u/2bc22234a58d?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)

(/p/3b3ea5b0758a?



utm_campaign=maleskine&utm_content=note&utm_medium=seo_notes&utm_source=recommendation)
巧用display:inline-block; (1) -----做菜单 (/p/3b3ea5b0758a?utm_ca...

案例：用a 标签做菜单 优点： 1.不脱离文档流。 2.减少层级嵌套关系。 坑----内联元素代码换行写浏览器会把换行当作空格来处理，会造成内联元素之间有间距 解决的方法： 1.给内联元素的父级元素设置font-size:0;

 RanAzhe (/u/23fe76108caf?

utm_campaign=maleskine&utm_content=user&utm_medium=seo_notes&utm_source=recommendation)